

Lane Detection

Lane Detection

1. Course Content
2. Run Example Program
5. Lane Line Detection Principles and Source Code Analysis
 - 5.1 Lane Line Detection Principles
 - 5.3 Lane Line Detection Source Code
 - 5.3.1 YOLOv11 Model Inference Source Code
 - 5.3.2 Main Program for Lane Line Detection Processing

1. Course Content

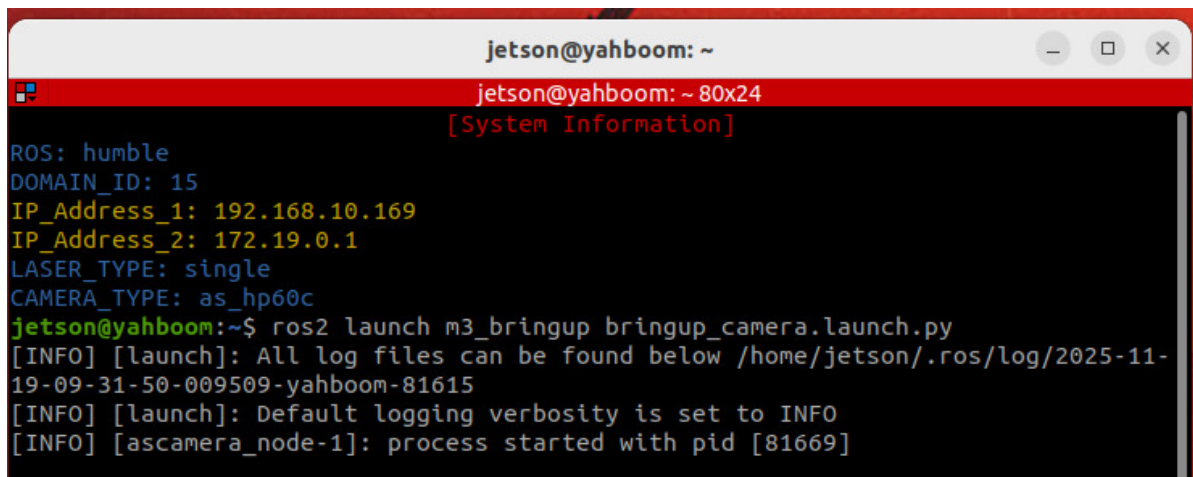
Basic: Start the autonomous driving program and observe lane line recognition results.

Advanced: Master lane line detection principles combined with debug mode.

2. Run Example Program

- Start camera node

```
ros2 launch m3_bringup bringup_camera.launch.py
```

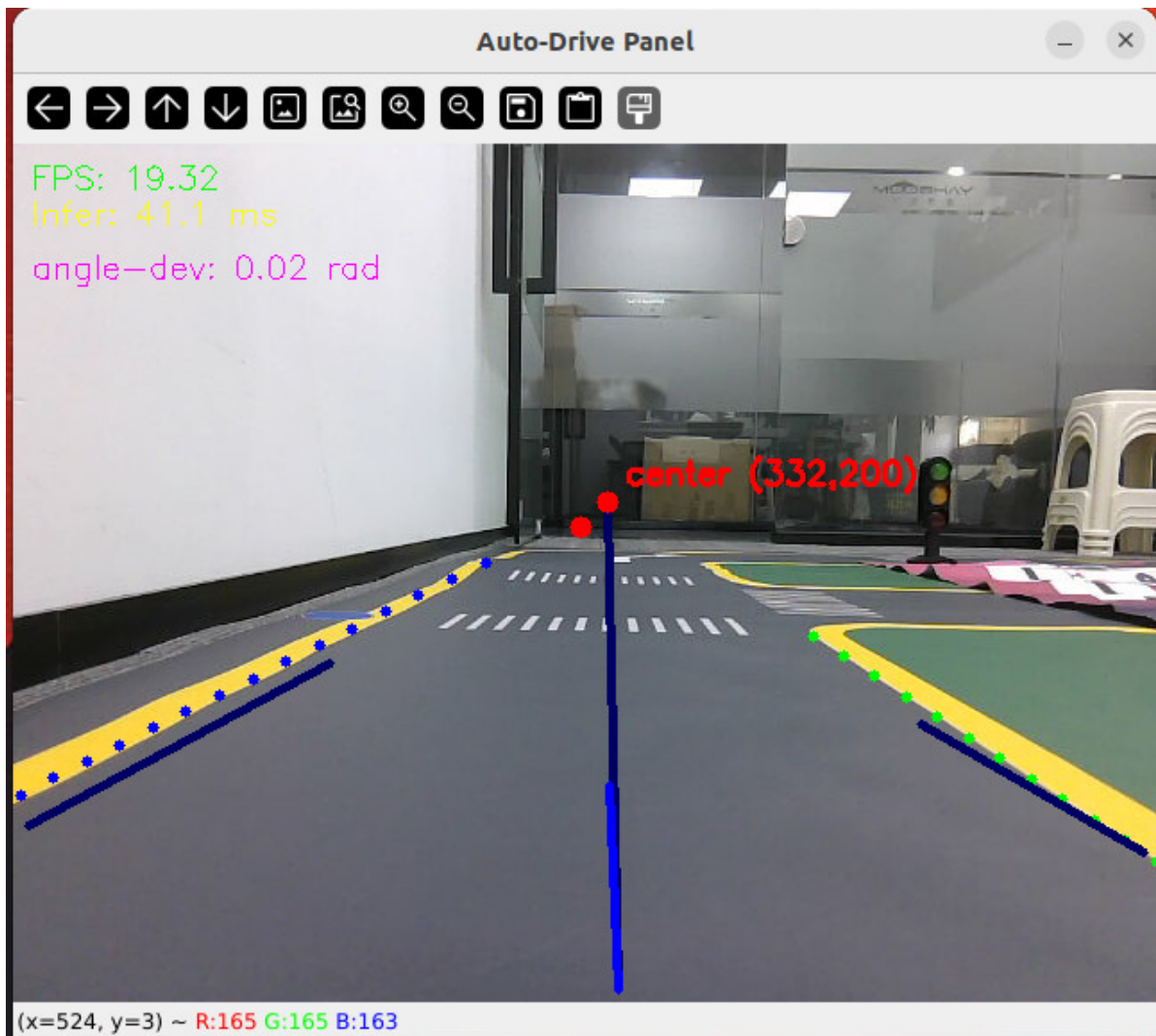


```
jetson@yahboom: ~  
jetson@yahboom: ~ 80x24  
[System Information]  
ROS: humble  
DOMAIN_ID: 15  
IP_Address_1: 192.168.10.169  
IP_Address_2: 172.19.0.1  
LASER_TYPE: single  
CAMERA_TYPE: as_hp60c  
jetson@yahboom:~$ ros2 launch m3_bringup bringup_camera.launch.py  
[INFO] [launch]: All log files can be found below /home/jetson/.ros/log/2025-11-19-09-31-50-009509-yahboom-81615  
[INFO] [launch]: Default logging verbosity is set to INFO  
[INFO] [ascamera_node-1]: process started with pid [81669]
```

- Start autonomous driving node

```
ros2 run auto_drive auto_drive
```

- The blue dotted line segment is the left lane line, and the green dotted line segment is the right lane line



- Lane line debug mode

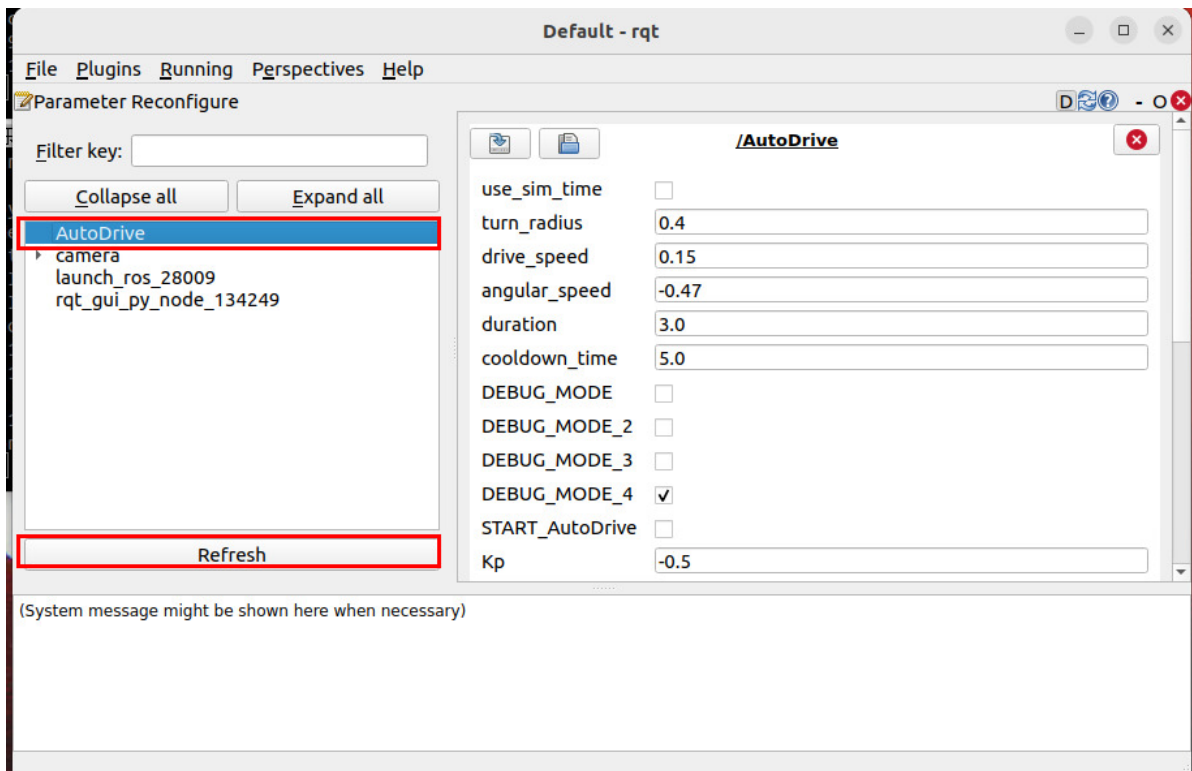
[!NOTE]

When you need to observe the YOLO detection boxes of lane lines to debug the effect, you can start the lane line debug mode

- Start dynamic parameter configuration node

```
ros2 run rqt_reconfigure rqt_reconfigure
```

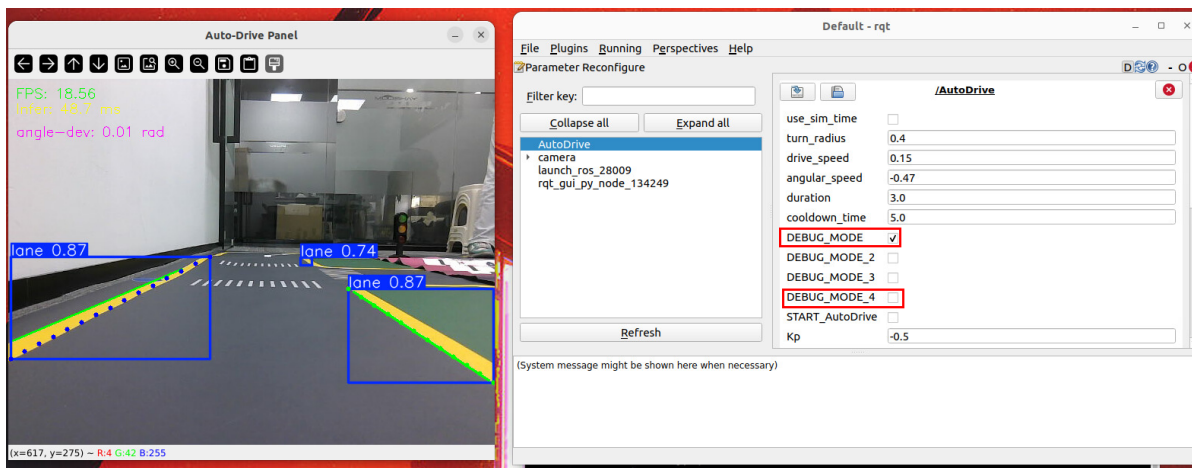
- Click AutoDrive to enter the parameter control panel. If there is no `AutoDrive` on the left side when you first open it, click `Refresh` below



- Among them, `DEBUG_MODE` is the switch to display lane line recognition boxes, and `DEBUG_MODE_4` is the switch to display lane reference lines. We can first turn off `DEBUG_MODE_4` and turn on `DEBUG_MODE` to separately observe the lane line detection results.

[!TIP]

The blue recognition box with the lane title in the image is the lane line recognition result. The green solid line in the detection box is the line segment detected by Hough, which is used to assist in determining the left and right lane lines.



5. Lane Line Detection Principles and Source Code Analysis

5.1 Lane Line Detection Principles

1. Lane line detection first uses the YOLOv11 object detection model to detect the distribution area of lane lines

[!NOTE]

- If you need to train a custom lane line detection model, please refer to section 5 of this chapter course: [05-YOLOv11 Model Training](#)

2. Then through Hough line detection, find the longest line segment in the detection box, and determine the left and right lane lines by judging the line slope

[!TIP]

- Lane line detection can also be achieved through YOLOv11's segmentation model and OBB rotation detection model, or other types of recognition models, but on Jetson devices, the object detection model's inference speed is faster and real-time performance is better.
- The YOLOv11 object detection model can only detect the distribution of lane lines but cannot directly distinguish between left and right lane lines, because the features of left and right lane lines are extremely similar. If the data is annotated as left and right lane lines during the data annotation stage, the probability of misjudgment between left and right lane lines will be very high during detection. Therefore, this method uses YOLOv11 + OpenCV Hough detection combination to comprehensively determine left and right lane lines.

5.3 Lane Line Detection Source Code

- YOLOv11 inference source code:

```
~/yahboomM3_ws/code_ws/src/auto_drive/auto_drive/utis/yolov11_infer.py
```

- auto_drive.py main program source code:

```
~/yahboomM3_ws/code_ws/src/auto_drive/auto_drive/auto_drive.py
```

5.3.1 YOLOv11 Model Inference Source Code

- YOLO_MODEL in yolov11_infer.py is the base class for YOLO detection, used to extend different YOLO recognition model programs

```

class YOLO_MODEL(ABC):
    '''Model base class'''
    def __init__(self):
        self.yolo_model=None
    @abstractmethod
    def get_infer_result(self,source):
        """Return the model recognition result"""
        pass
    @abstractmethod
    def init_yolo_engine(self):
        """Load model"""
        pass

```

- Lane_Detect is a subclass that inherits from YOLO_MODEL, containing various methods for lane line detection

```

class Lane_Detect(YOLO_MODEL):
    """Lane line detection model"""
    def __init__(self,config_file_path:str,
                 midline_file_path:str):
        with open(config_file_path, "r") as file:
            full_config = yaml.safe_load(file)
            self.config_param = full_config.get("line_detection", {})
        with open(midline_file_path, "r") as file:
            self.point_config = yaml.safe_load(file)
    -----

```

- get_infer_result is a method to infer images and obtain lane line detection results

```

def get_infer_result(self,source,stream:bool=True):
    '''Obtain the reasoning result of a single frame image'''
    result =
self.yolo_model(source,stream=stream,conf=self.conf,iou=self.iou,rect=self.rect,
half=self.half,

device=self.device,batch=self.batch,max_det=self.max_det,augment=self.augment,
verbose=self.verbose,classes=self.classes)

    return result

```

- The classify_lane_direction method uses the slope of the longest line detected by Hough from the YOLO recognition box to assist in determining the left and right lane lines in the YOLO box

```

def classify_lane_direction(self, frame, box, debug=False):
    """
    Hough detection determines the direction of lane lines
    """

    xmin, ymin, xmax, ymax = map(int, box)

```

```

roi = frame[ymin:ymax, xmin:xmax]
if roi.size == 0:
    return None

gray = cv2.cvtColor(roi, cv2.COLOR_BGR2GRAY)
edges = cv2.Canny(gray, 50, 150)

lines = cv2.HoughLinesP(
    edges, 1, np.pi / 180,
    threshold=30,
    minLineLength=20,
    maxLineGap=10
)

if lines is None:
    return None

#Filter out horizontal and vertical lines
filtered = []
for line in lines:
    x1, y1, x2, y2 = line[0]
    dx = x2 - x1
    dy = y2 - y1

    if dx == 0:
        continue

    slope = dy / dx

    if abs(slope) < 0.03: # Too level
        continue
    if abs(slope) > 10: # Too vertical
        continue

    filtered.append(line)

if len(filtered) == 0:
    return None

longest = max(
    filtered,
    key=lambda l: math.hypot(l[0][2]-l[0][0], l[0][3]-l[0][1])
)

x1, y1, x2, y2 = longest[0]

if debug:
    px1, py1 = xmin + x1, ymin + y1
    px2, py2 = xmin + x2, ymin + y2
    cv2.line(frame, (px1, py1), (px2, py2), (0,255,0), 2)

dx = x2 - x1
dy = y2 - y1
slope = dy / dx
if slope < 0:
    return "left"
else:
    return "right"

```

- The `__draw_dotted_line` method is used to draw the recognized detection boxes into visualized lane lines (dotted line segments)

```
def __draw_dotted_line(self, image, start_point, end_point, color=(255, 0, 0),
thickness=3, dot_spacing=20):
    """Draw dotted line segments composed of points"""

    # Calculate the distance and direction between two points
    dx = end_point[0] - start_point[0]
    dy = end_point[1] - start_point[1]
    distance = np.sqrt(dx**2 + dy**2)

    num_dots = int(distance / dot_spacing)
    points = []
    for i in range(num_dots + 1):
        t = i / num_dots if num_dots > 0 else 0
        x = int(start_point[0] + t * dx)
        y = int(start_point[1] + t * dy)
        points.append((x, y))
    for point in points:
        cv2.circle(image, point, thickness, color, -1)
    return image
```

5.3.2 Main Program for Lane Line Detection Processing

- `image_callback` is the callback function for subscribing to the camera topic, used to receive real-time camera images, then convert them to OpenCV format images and push them into the cache queue of the `Lane_Detect` class.

```
def image_callback(self, msg: Image)->None:
    '''The ROS image callback function converts the ROS image to OpenCV
    format and adds it to the processing queue.
    '''
    cv_image = self.bridge.imgmsg_to_cv2(msg, desired_encoding='bgr8')
    self.lane_detect.put_image(cv_image.copy())
```

- The `handle_lane` method is responsible for taking image frames from the cache queue of the `Lane_Detect` class for processing. The processing flow is: 1. Take image frame from queue → 2. Model inference to get lane line detection results → 3. Classify left and right lane candidate boxes → 4. Filter optimal left and right lane boxes → 5. Visualize results → 6. Loop to wait for next frame

The program for classifying left and right lane lines is as follows:

```

for res in results:
    boxes = res.boxes # All detection boxes for a single frame (xyxy format:
xmin, ymin, xmax, ymax)
    if self.DEBUG_MODE: frame = res.plot() # Debug mode: draw all detection
boxes to image
    if boxes is not None:
        for i in range(len(boxes)):
            box_coords = boxes.xyxy[i].tolist() # Coordinates of a single
detection box (top-left + bottom-right)
            # Key: call classification function to determine if this box belongs
to left lane/right lane
            lane = self.lane_detect.classify_lane_direction(frame, box_coords,
debug=self.DEBUG_MODE)
            if lane == "left":
                left_candidates.append(box_coords) # Add to left lane
candidates
            elif lane == "right":
                right_candidates.append(box_coords) # Add to right lane
candidates

```

```

def handle_lane(self)->None:
    '''Processing lane line recognition detection results'''

    while True:
        try:
            frame = self.lane_detect.image_queue.get(timeout=0.1)
        except queue.Empty:
            continue
        infer_start = time.time()
        results =self.lane_detect.get_infer_result(frame, stream=True)
        #Initialize default values
        left_box = None
        right_box = None
        left_candidates = []
        right_candidates = []

        for res in results:
            boxes = res.boxes # Boxes object for bounding box outputs
            if self.DEBUG_MODE: frame = res.plot()
            if boxes is not None:
                for i in range(len(boxes)):
                    box_coords = boxes.xyxy[i].tolist() # Get the
bounding box coordinates (xyxy format: xmin, ymin, xmax, ymax)

            lane=self.lane_detect.classify_lane_direction(frame,box_coords,debug=self.DEBUG
_MODE)

            if lane=="left":
                left_candidates.append(box_coords)
            elif lane=="right":
                right_candidates.append(box_coords)

        if len(left_candidates)>0 :
            left_box = max(left_candidates, key=lambda box: box[3]) # Select the
bounding box closest to the bottom (with the largest ymax).

```

```

        if len(right_candidates)>0:
            right_box = max(right_candidates, key=lambda box: ((box[2] -
box[0])**2 + (box[3] - box[1])**2)**0.5)

        frame=self.__handle_lane_detect_result(left_box,right_box,frame)
        infer_end = time.time()

        self.frame_count_1 += 1
        curr_time = time.time()
        time_diff = curr_time - self.prev_time_1
        if time_diff >= 1.0:
            self.fps_1 = self.frame_count_1 / time_diff
            self.frame_count_1 = 0
            self.prev_time_1 = curr_time
        cv2.putText(frame,f"FPS: {self.fps_1:.2f}",(10,
25),cv2.FONT_HERSHEY_SIMPLEX,0.6,(0, 255, 0),1,)

        infer_time = (infer_end - infer_start) * 1000
        cv2.putText(frame, f"Infer: {infer_time:.1f} ms", (10,
45),cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 255), 1)
        cv2.imshow("Auto-Drive Panel", frame)
        cv2.waitKey(1)

```